

15 장 학습 과제

과제 1. 안전한 정수 입력

정수를 입력받는 `ReadInteger()` 함수를 작성한다. 문자열이나 실수를 입력하면 오류 상태와 현재 행의 나머지를 제거하고 다시 입력받는다. 최솟값과 최대값을 인수로 전달해 범위를 벗어난 값도 다시 입력받도록 확장한다.

과제 2. 사용자 정의 타입 입출력

`Item` 구조체에 `id`, `name`, `count`, `price`를 정의하고 `operator<<`와 `operator>>`를 구현한다. 입력 과정에서 일부 필드가 실패하면 기존 객체의 값이 변경되지 않아야 한다. `id <= 0`, `count < 0`, `price < 0`은 `failbit`를 설정해 거부한다.

과제 3. 문자열 전체 파싱

문자열을 `double` 하나로 변환하는 `ParseDouble()` 함수를 작성한다. 앞뒤 공백은 허용하지만 숫자 뒤에 다른 문자가 남으면 실패해야 한다. 반환 타입은 `std::optional<double>`을 사용한다.

과제 4. 형식화된 목록 출력

여러 `Item`을 이름, 수량, 단가, 총액 열로 출력한다. 스트림 조정자를 사용하는 버전과 `std::format()`을 사용하는 버전을 각각 작성한다. 이름은 왼쪽 정렬, 숫자는 오른쪽 정렬하고 금액은 소수점 이하 두 자리로 표시한다.

과제 5. 텍스트 파일 저장과 불러오기

`PlayerData` 배열을 텍스트 파일에 저장하고 다시 읽는다. 한 행에 하나의 객체를 기록하고 잘못된 행은 행 번호와 함께 오류로 출력한다. 파일을 열지 못한 경우, 형식 오류, 장치 읽기 오류를 구분한다.

과제 6. 단순 CSV와 제한 조건

쉼표로 구분된 아이템 파일을 읽어 `std::vector<Item>`으로 변환한다. 각 행의 필드 수와 숫자 변환 성공 여부를 검사한다. 이름에 쉼표와 줄바꿈이 들어가지 않는다는 제한을 문서화하고, 일반 CSV와 다른 점을 설명한다.

과제 7. 바이너리 레코드 형식

매직 값, 버전, 아이템 개수와 아이템 필드를 포함하는 바이너리 형식을 설계한다. 정수는 리틀 엔디언으로 기록하고 문자열은 길이와 UTF-8 바이트를 기록한다. 읽을 수 있는 최대 아이템 개수와 최대 문자열 길이를 정해 파일 손상에 대비한다.

과제 8. 파일 버전 변환

버전 1에는 `id`, `name`, `hp`, 버전 2에는 `level` 필드가 추가된 플레이어 파일을 구현한다. 버전 1 파일을 읽으면 `level`에 기본값을 적용한다. 알 수 없는 버전은 오류로 처리한다.

과제 9. 파일 검색

지정한 디렉토리를 재귀적으로 순회해 확장자가 `.png`인 파일을 찾는다. 결과 경로를 정렬하고 파일 크기와 함께 출력한다. 접근 권한 오류는 기록한 뒤 다른 항목의 순회를 계속한다.

과제 10. 안전한 임시 파일 저장

원본 파일을 직접 덮어쓰지 않고 임시 파일에 저장한 뒤 교체하는 함수를 작성한다. 임시 파일 기록 실패, 이름 변경 실패와 기존 백업 파일 처리 정책을 구분한다.

과제 11. 난수 주사위

`std::mt19937`과 `std::uniform_int_distribution`으로 주사위를 백만 번 던지고 각 눈의 출현 횟수를 출력한다. `engine() % 6 + 1` 방식의 결과와 비교하고 분포 차이를 분석한다.

과제 12. 재현 가능한 던전 생성

정수 시드를 받아 방의 위치와 아이템 배치를 생성한다. 같은 시드에서는 같은 결과가 만들어져야 한다. 실행 로그에 시드를 기록하고 사용자가 이전 시드를 입력해 같은 던전을 다시 생성할 수 있도록 구성한다.

과제 13. 무작위 셔플과 추첨

참가자 이름을 저장한 벡터를 `std::shuffle()`로 섞고 중복 없이 당첨자를 선택한다. 빈 목록, 당첨 인원이 참가자 수보다 많은 경우를 처리한다.

과제 14. 실행 시간 측정

같은 데이터에 대해 선형 검색과 정렬 후 이진 탐색의 실행 시간을 측정한다.

`steady_clock`을 사용하고 여러 번 반복해 평균 시간을 구한다. 데이터 생성 시간은 검색 시간에서 제외한다.

과제 15. 쿨다운과 고정 업데이트

`std::chrono` 타입을 사용하는 쿨다운 클래스를 구현한다. 별도의 고정 업데이트 예제에서는 프레임 시간을 누적하고 60분의 1초 간격으로 상태를 갱신한다. 한 프레임에서 수행할 최대 업데이트 횟수를 제한한다.

참고. C++23의 `std::print`

C++23의 `std::print()`는 `std::format` 형식 문자열을 사용해 표준 출력에 직접 기록한다.

```
#include <print>
int main()
{
    std::print("Name: {}, Level: {}\n", "Knight", 12);
}
```

`std::print()`는 줄바꿈을 자동으로 추가하지 않는다. `std::println()`은 출력 끝에 줄바꿈을 추가한다.

`std::println("Name: {}, Level: {}", "Knight", 12);`

형식화된 결과를 먼저 문자열로 만들 필요가 없다.

```
std::cout << std::format(
    "Score: {}\n",
    1500);

std::print("Score: {}\n", 15
```

FILE*에도 출력할 수 있다.

```
#include <cstdio>
#include <print>

int main()
{
    std::FILE* file = std::fopen("log.txt", "w");

    if (file == nullptr)
    {
        return 1;
    }

    std::println(file, "Player {} joined", "Knight");
    std::fclose(file);
}
```

std::print와 std::println은 C++23 기능이다. C++20을 기준으로 하는 코드에서는 스트림 출력이나 std::format()으로 만든 문자열을 사용한다.

